

Database & Vector-Store Schema

The system uses two local stores:

1. **ChromaDB** (data/chroma_db/) — the vector index that powers semantic + hybrid search.
2. **SQLite** (data/ingestion.db) — the operational ingestion log behind GET /documents.

1. ChromaDB collection: documents

Created in backend/vectorstore.py with cosine space (metadata={"hnsw:space": "cosine"}). One record per text chunk.

Field	Type	Source	Filterable	Notes
id	string	<document_id>_p<page>_c<chunk>	(by id)	Stable, unique per chunk
embedding	float[]	BGE-M3 dense vector (normalised)	—	Cosine similarity ranking
document	string	chunk text	—	Returned as the source snippet
metadata.document_id	string (uuid)	generated at upload		Groups chunks of one file
metadata.filename	string	uploaded filename		Shown in citations
metadata.document_type	string	manual (upload form)	\$eq	e.g. invoice/report/letter/form/other
metadata.document_date	string YYYY-MM-DD or ""	manual (upload form)	\$gte/\$lte	Range filter via lexicographic ISO order
metadata.language	string bn/en/mixed	per-chunk detection	\$eq	Unicode-range heuristic
metadata.page_number	int	OCR page index (0-based)		Citation
metadata.chunk_index	int	chunk position in page		Ordering
metadata.upload_timestamp	string ISO-8601	generated at upload		Provenance

ChromaDB metadata values cannot be null; an absent `document_date` is stored as "" and surfaced back to the API as null.

Hybrid where clause

Built dynamically in `build_where()`:

```
# one condition -> flat dict
{"language": {"$eq": "bn"}}

# multiple conditions -> explicit $and (required by current ChromaDB)
{"$and": [
  {"document_type": {"$eq": "invoice"}},
  {"language": {"$eq": "bn"}},
  {"document_date": {"$gte": "2024-01-01"}},
  {"document_date": {"$lte": "2024-12-31"}},
]}
```

Filters with no value are omitted; if nothing is present, `where` is dropped entirely (Chroma rejects `where={}`). The filter is a **pre-filter**: vector ranking only runs over chunks that already satisfy it.

2. SQLite table: ingestion_log

Defined in `backend/database.py`.

```
CREATE TABLE ingestion_log (
  document_id      TEXT PRIMARY KEY,
  filename         TEXT NOT NULL,
  upload_timestamp TEXT NOT NULL,
  page_count       INTEGER,
  chunk_count      INTEGER,
  languages_detected TEXT,          -- JSON array, e.g. ["bn","en"]
  document_date    TEXT,          -- YYYY-MM-DD or NULL
  document_type    TEXT,
  status           TEXT NOT NULL  -- 'pending'|'processing'|'done'|'error'
);
```

Column	Type	Purpose
document_id	TEXT PK	UUID, links to ChromaDB metadata.document_id
filename	TEXT	Original upload name
upload_timestamp	TEXT	ISO-8601 UTC ingestion time
page_count	INTEGER	Pages OCR'd
chunk_count	INTEGER	Chunks produced & embedded
languages_detected	TEXT	JSON array of distinct chunk languages
document_date	TEXT	Manual metadata (date the doc pertains to)
document_type	TEXT	Manual metadata category
status	TEXT	Lifecycle: processing → done /

Column	Type	Purpose
		error

The two stores join on `document_id`: SQLite is the human-readable catalogue, ChromaDB holds the searchable vectors.